DevDocs — Chat Interface

1. Purpose

The Chat Interface is the primary interaction point between the user and DevDocs.

The goal is not to build a generic chatbot.

DevDocs should provide a **developer-focused conversational interface for understanding a software project**, where the assistant explains the project using project-scoped knowledge and provides references back to the original files.

The core experience is:

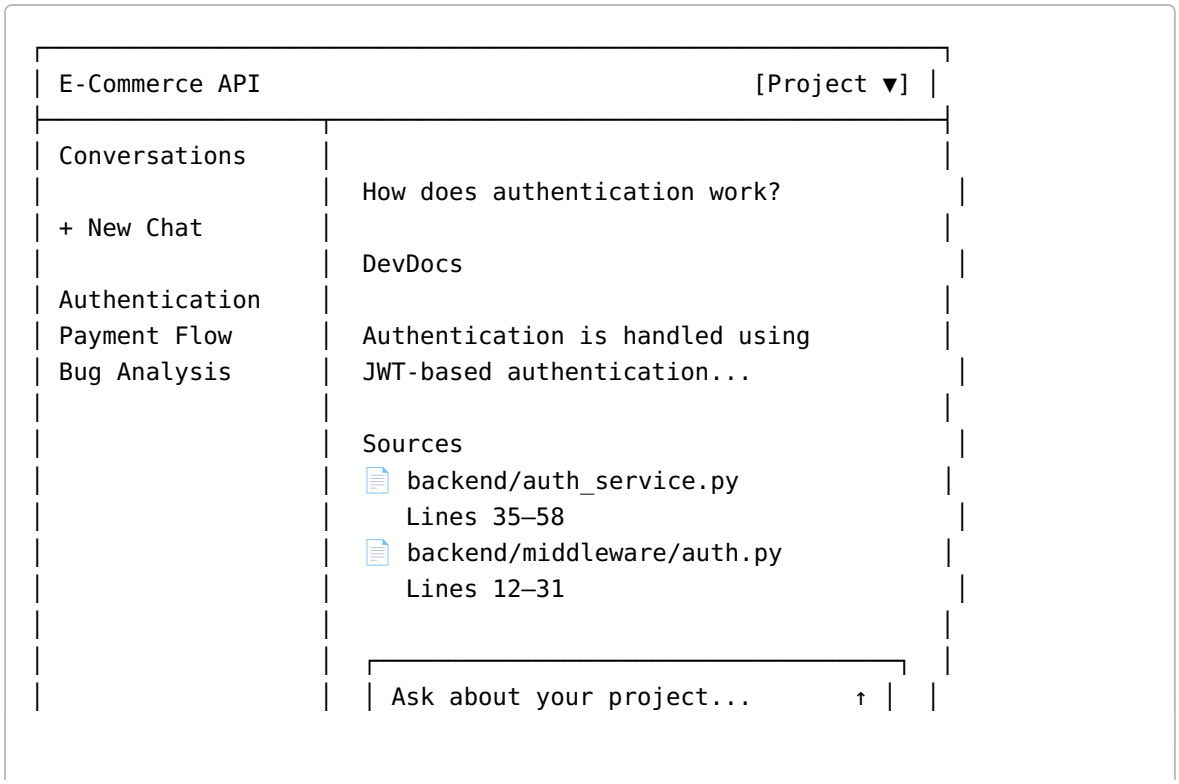
> **Ask a question → DevDocs understands the project → explains the answer → shows where the information came from.**

The assistant should primarily provide explanations rather than dumping retrieved code or document chunks into the response.

---

2. Overall UI

The chat interface consists of four main areas:



```
│                    │  └──────────────────────────────────────┘  │
└────────────────────┴────────────────────────────────────────────┘
```

The four main areas are:

1. Project Header
2. Conversation Sidebar
3. Chat Area
4. Message Composer

---

3. Project Header

The header identifies the project currently being investigated.

Example:

```
E-Commerce API
```

The chat is always associated with the currently selected project.

The frontend maintains the current project context:

```
current_project_id = 101
```

The backend uses this project ID to ensure that RAG retrieval only searches knowledge belonging to that project.

Project isolation is therefore enforced by the application/retrieval layer rather than by the LLM.

---

4. Conversation Sidebar

The sidebar contains the user's conversations for the current project.

Example:

```
Conversations

+ New Chat

Authentication Flow
Payment Processing
```

```
Bug Analysis
Order Creation
```

V1 actions

The user can:

- Start a new conversation
- Switch between existing conversations
- Delete a conversation

The user cannot rename conversations in V1.

---

5. Starting a New Conversation

`+ New Chat` opens a blank chat window.

It should initially be treated as a **temporary new-chat state** rather than immediately creating a permanent conversation in the database.

Example:

```
Start a conversation

Ask anything about your project

┌──────────────────────────────────────────────┐
│ Ask something about your project...      ↑ │
└──────────────────────────────────────────────┘
```

No LLM call is made.

No RAG retrieval is performed.

No conversation is persisted yet.

If the user opens a new chat and navigates away without sending anything, DevDocs should not create an empty conversation such as:

```
New Chat
```

The conversation is created only when the user sends the first message.

---

6. First Message Lifecycle

When the user sends the first message:

```
User enters question
        ↓
Validate message
        ↓
Display user message immediately
        ↓
Create conversation
        ↓
Generate conversation title
        ↓
Save user message
        ↓
Retrieve project-scoped knowledge
        ↓
Build conversation context
        ↓
Call LLM
        ↓
Save assistant response
        ↓
Display response + sources
```

Example question:

```
How does authentication work?
```

The conversation title can be generated using simple application-side logic.

Example:

```
How does authentication work?
        ↓
Authentication Flow
```

Another example:

```
Where is payment processing implemented?
        ↓
Payment Processing
```

V1 should not require an additional LLM call just to generate a conversation title.

This avoids unnecessary latency and token usage.

The title is generated once from the first question and remains stable.

---

7. Message Validation

Before sending a message:

- Reject empty messages.
- Reject whitespace-only messages.
- Apply a reasonable maximum input length.

A practical V1 limit can be approximately:

```
4,000–8,000 characters
```

The exact limit can be finalized during implementation.

---

8. Immediate User Feedback

The user's message should appear immediately after clicking Send.

The frontend should not wait for the LLM response before displaying it.

Example:

```
User:

How does authentication work?


DevDocs:

Thinking...
```

While the backend is processing the request, the input/send controls should be temporarily disabled.

This prevents multiple concurrent messages from creating ordering and state-management problems in V1.

Future versions can support:

- Concurrent requests
- Request cancellation

- Streaming responses
- Retry controls

---

9. Backend Message Flow

Conceptually, the frontend sends:

```
POST /conversations/{conversation_id}/messages
```

Request:

```
{
  "content": "How does authentication work?"
}
```

The backend already knows:

```
conversation_id
project_id
```

The backend then performs:

```
Save user message
        ↓
Load conversation context
        ↓
Retrieve project-scoped RAG results
        ↓
Build prompt/context
        ↓
Call LLM
        ↓
Receive response
        ↓
Save assistant message
        ↓
Return response + source metadata
```

---

10. Project-Scoped RAG

The assistant must only retrieve information belonging to the current project.

For example:

```
Project A
project_id = 101

Project B
project_id = 102
```

If the user is currently inside Project A:

```
project_id = 101
```

RAG retrieval must filter:

```
project_id == 101
```

before the retrieved chunks are passed to the LLM.

The LLM should never be responsible for deciding which project it is allowed to access.

The application controls the knowledge boundary.

---

11. Conversation Context

DevDocs stores the complete conversation history in the database.

However, the complete history should not always be sent to the LLM.

A conversation could eventually contain hundreds of messages, creating:

- Large prompts
- Higher token usage
- Higher latency
- Context pressure
- Irrelevant historical information

Therefore, the LLM context should be limited.

The conceptual structure is:

```
Older conversation
        ↓
Conversation summary
```

```
Recent messages
        ↓
Limited recent context

Current question

        +

Project-scoped RAG results

        ↓

LLM
```

Important distinction:

> **Restoring a conversation is not the same as sending the entire conversation history to the LLM.**

The UI can display the complete history while the backend provides only the relevant context to the model.

---

12. Switching Conversations

When the user clicks an existing conversation:

```
Authentication Flow
```

DevDocs loads the stored messages from the database.

No RAG retrieval is required merely to open the conversation.

No LLM call is required.

No new conversation is created.

The UI simply reconstructs the stored conversation.

Example:

```
Authentication Flow

User:
How does authentication work?

DevDocs:
```

```
Authentication is handled using JWT...

User:
Where is the token generated?

DevDocs:
The token is generated...
```

The user can then continue the conversation.

When they send a new message, the normal conversation-context + project-scoped RAG flow is executed.

---

13. Deleting a Conversation

Each conversation can have a menu:

```
Authentication Flow     ⋮
```

The V1 menu contains:

```
Delete
```

Rename is intentionally excluded from V1.

A confirmation should be displayed before deletion.

Example:

```
Delete "Authentication Flow"?

This will permanently delete this conversation
and its messages.

[Cancel] [Delete]
```

Deleting a conversation removes:

```
Conversation
    ├── Messages
    └── Conversation metadata
```

It does **not** remove:

```
Project
Files
Chunks
Embeddings
Ingestion data
Project knowledge
```

Therefore:

> **Deleting a conversation ≠ deleting project knowledge.**

---

14. Deleting the Currently Open Conversation

If the user deletes the conversation they are currently viewing:

If other conversations exist

Open the most recently updated remaining conversation.

If no conversations remain

Show the blank new-chat state:

```
Start a conversation

Ask anything about your project
```

Deleting another conversation that is not currently open does not affect the active chat.

---

15. Assistant Response Design

The assistant response should be designed as a **developer explanation**, not simply as a string containing retrieved chunks.

The response should support:

- Natural-language explanations
- Markdown
- Lists
- Headings
- Small code snippets when useful
- Source references

The default behavior is:

> **Explain first, cite the source, don't dump the code.**

---

16. Example Assistant Response

User:

```
How does authentication work?
```

DevDocs:

```
Authentication is handled using JWT-based authentication.

The flow is:

1. The user submits their credentials.
2. The authentication service validates the credentials.
3. A JWT token is generated.
4. The client sends the token with subsequent requests.
5. Authentication middleware validates the token before allowing
   access to protected endpoints.

Sources

📄 backend/services/auth_service.py
   Lines 35–58

📄 backend/middleware/auth.py
   Lines 12–31

📄 backend/routes/login.py
   Lines 10–29
```

The assistant does not automatically display the entire contents of those files.

---

17. Code Snippets

The rule should not be:

> Never show code.

Instead:

> **Don't show code by default. Show a small relevant snippet when it helps answer the question or when the user explicitly asks for it.**

For example, if the user asks:

```
Show me where the JWT token is generated.
```

DevDocs could respond:

```
The JWT token is generated after the user's password
has been successfully verified.

backend/services/auth_service.py — Lines 42–45

token = create_access_token(user.id)
```

The application should avoid dumping large retrieved chunks or entire files.

---

18. Source Citations

Source citations are an important part of DevDocs.

They provide:

- Transparency
- Trust
- Traceability
- A way for developers to verify the answer
- A path to inspect the original implementation

The citation should be generated from **source metadata stored during ingestion/retrieval**, not invented by the LLM.

---

19. Code File Citations

For source code files, every chunk should store:

```
start_line
end_line
```

Example:

```
file_path: backend/services/auth_service.py
start_line: 35
end_line: 58
```

The user-facing citation becomes:

```
📄 backend/services/auth_service.py
   Lines 35–58
```

The same start/end range stored with the retrieved chunk is used for the citation.

---

20. Why Code Files Need Line Numbers

Line numbers are particularly useful for source code because developers naturally navigate code using line locations.

Example:

```
backend/services/auth_service.py — Lines 35–58
```

This gives the user an immediate location to inspect.

It also provides a foundation for future functionality where clicking the citation could open the file at that location.

---

21. Other File Types

Line numbers should **not** be forced onto every type of document.

Different file types should use the most natural location reference.

Code

```
backend/auth.py — Lines 42–58
```

PDF

```
architecture.pdf — Page 7
```

Markdown

```
README.md — Authentication Setup
```

Text

Use a section or other meaningful structural reference where available.

DOCX

Use an appropriate structural location such as a heading/section where available.

The principle is:

> **Use the native structural location that makes sense for the file type.**

---

22. Chunk Metadata

The ingestion pipeline should preserve source-location information.

For code chunks:

```
{
  "project_id": 101,
  "file_id": 501,
  "chunk_id": 1001,
  "file_path": "backend/services/auth_service.py",
  "file_type": "python",
  "start_line": 35,
  "end_line": 58,
  "content": "..."
}
```

For PDF chunks:

```
{
  "project_id": 101,
  "file_id": 502,
  "chunk_id": 1002,
  "file_path": "docs/architecture.pdf",
  "file_type": "pdf",
  "page_number": 7,
  "content": "..."
}
```

For Markdown:

```
{
  "project_id": 101,
  "file_id": 503,
  "chunk_id": 1003,
  "file_path": "README.md",
  "file_type": "markdown",
  "section": "Authentication Setup",
  "content": "..."
}
```

This metadata is stored alongside the chunk and embedding.

---

23. Citation Generation

The LLM should not be trusted to invent citation locations.

Instead:

```
Retrieved chunk
       ↓
Chunk metadata
       ↓
file_path
start_line / end_line
page_number
section
       ↓
LLM generates explanation
       ↓
Backend attaches source metadata
       ↓
Frontend renders citations
```

This makes citations deterministic and reliable.

---

24. V1 Citation Precision

For V1, the citation should refer to the **retrieved chunk's location**.

Example:

```
backend/auth_service.py — Lines 35–58
```

We should not pretend that we know the exact lines responsible for every individual sentence unless the system actually performs precise span-level attribution.

Therefore, V1 uses:

> **Chunk-level source location**

rather than:

> **Exact sentence-level source location**

Future versions can make this more precise.

---

25. Future Source Experience

A future version can make citations interactive.

Example:

```
📄 backend/services/auth_service.py
   Lines 35–58
```

Clicking the citation could open a code viewer:

```
backend/services/auth_service.py

35  def authenticate_user(...):
36      ...
37      ...
...
42      token = create_access_token(user.id)
...
58      return token
```

The relevant location could be highlighted.

Possible future enhancements:

- Clickable source references
- Open file at exact line
- Code highlighting
- Source preview
- Relevant snippet preview

- More precise span-level attribution
- Function/class-level citations
- Dependency navigation

These are future enhancements and do not need to complicate the initial implementation.

---

26. Complete Chat Architecture

The complete conceptual flow is:

```
                    DEV DOCS CHAT

User
 │
 │ Question
 ↓
React Chat Interface
 │
 │ POST /conversations/{id}/messages
 ↓
FastAPI
 │
 ├── Validate message
 │
 ├── Save user message
 │
 ├── Load conversation context
 │
 ├── Project-scoped RAG
 │      │
 │      ├── Embedding/search
 │      ├── project_id filter
 │      └── Retrieve relevant chunks
 │
 ├── Build LLM context
 │      │
 │      ├── System instructions
 │      ├── Conversation summary
 │      ├── Recent messages
 │      ├── Current question
 │      └── Retrieved project sources
 │
 ├── Call LLM
 │
 ├── Save assistant response
 │
 └── Return response + source metadata
          │
          ↓
```

```
React Chat Interface
 │
 ├── Markdown explanation
 ├── Code snippets when useful
 └── Source citations
        │
        ├── Code → Lines
        ├── PDF → Page
        └── Docs → Section
```

---

27. Core Design Principles

The Chat Interface follows these principles:

1. Explain, don't dump

The assistant should explain the project rather than dumping retrieved chunks.

2. Cite, don't make the user trust blindly

Every RAG-based answer should provide useful source references.

3. Don't let the LLM invent citations

Source locations come from application metadata.

4. Project isolation is mandatory

The assistant only receives information from the current project.

5. Full history can be stored without full history being sent to the LLM

The UI and database can maintain complete conversations while the LLM receives optimized context.

6. Conversations and project knowledge are separate

Deleting a conversation must never delete the project's files, chunks, or embeddings.

7. Code is available when needed, not forced into every answer

Default experience is explanation + citations.

8. Use natural source locations

Code → lines
PDF → pages
Markdown → sections
Other documents → appropriate structural location.

28. V1 Scope

The V1 Chat Interface should include:

- Project-aware chat
- Conversation sidebar
- New Chat
- Multiple conversations per project
- Conversation switching
- Conversation deletion
- Blank new-chat state
- First-message conversation creation
- Automatic/simple conversation title
- User message validation
- Loading state
- Temporary input disabling while processing
- Project-scoped RAG
- Conversation context management
- Conversation summary + recent messages
- Markdown responses
- Small code snippets when appropriate
- Source citations
- Code line-number citations
- PDF page citations
- Document/Markdown section citations
- Source metadata stored with chunks
- Assistant message persistence

---

29. Future Enhancements

The following can be considered after V1:

- Streaming responses
- Cancel generation
- Retry failed responses
- Clickable source citations
- Integrated code viewer
- Open file at cited line
- Code highlighting
- Exact source-span attribution
- Function/class-level citations
- Source previews
- Message feedback
- Answer regeneration
- Suggested follow-up questions
- Conversation search
- Concurrent message handling

30. Final Product Experience

The final intended experience is:

```
Developer asks:

"How does authentication work?"
                ↓
DevDocs searches only the current project
                ↓
Relevant code/document chunks are retrieved
                ↓
Conversation context is added
                ↓
LLM generates a clear explanation
                ↓
DevDocs displays:

Authentication is handled using JWT...

Sources:
📄 backend/auth_service.py — Lines 35–58
📄 backend/middleware/auth.py — Lines 12–31
📄 README.md — Authentication Setup
```

The user gets:

Understanding + Evidence + Location

rather than simply:

Retrieved Text + LLM Response

This is what makes the Chat Interface feel like a genuine developer productivity tool rather than a generic RAG chatbot.